

Voice Control Object Tracking

1. Function Description

After the program starts, the robotic arm will move to the tracking pose. Hold the object you want to track and grasp, and appear it in the camera field of view. Use the mouse to draw a box around the selected object, and release the mouse to complete the selection. Say "Hi, yahboom" to the voice recognition module, and the speaker will announce "I am here". Then say "Tracking and picking up objects" to the voice module, and the speaker will announce "OK". Then the robotic arm starts tracking the selected object. Slowly move the object, and the robotic arm will also move along with it.

Note: When tracking an object, it needs to be moved slowly within the camera's field of view. This case will not perform grasping operations on the target, it only has the tracking function!

2. Startup and Operation

```
# Start camera and inverse kinematics
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start voice recognition and announcement
ros2 launch yahboom_speech speech.launch.py
# Start robotic arm control
ros2 run dofbot_voice_ctrl grasp_VC
# Start object tracking
ros2 run dofbot_voice_ctrl KCF_follow
```

3. Core Code Analysis

Object Tracking Node KCF_follow

Source code path:

/home/yahboom/dofbot_ws/src/dofbot_voice_ctrl/dofbot_voice_ctrl/KCF/KCF_follow.py

Image subscription callback

```
def ImageCallback(self, color_frame):
    # Convert ROS2 Image message to OpenCV BGR image format
    # 将ROS2 Image消息转换为OpenCV BGR图像格式
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'bgr8')

    # Create a copy of the image for drawing results
    # 创建图像副本用于绘制结果
    result_image = np.copy(rgb_image)

    # Capture key press for user interaction
    # 捕获按键以进行用户交互
    action = cv.waitKey(10) & 0xFF

    # Spacebar toggles the follow mode
    # 空格键切换跟随模式
    if action == 32:
        self.start_follow = True
```

```

# Resize the image to 640x480 resolution
# 将图像调整为640x480分辨率
result_image = cv.resize(result_image, (640, 480))

# Process the image
# 处理图像
result_frame = self.process(result_image, action)

# Check if object is detected and follow mode is active
# 检查物体是否被检测到且跟随模式是否激活
if self.cx != 0 and self.cy != 0 and self.start_follow == True:
    # Image center: (320, 240) for 640x480 resolution
    # 图像中心: (320, 240), 对应640x480分辨率
    # Threshold: 10 pixels (dead zone to avoid jitter)
    # 阈值: 10像素 (死区以避免抖动)
    if (abs(self.cx - 320) > 10 or abs(self.cy - 240) > 10):
        # Call XY tracking function to adjust arm position
        # 调用XY跟踪函数以调整机械臂位置
        self.XY_track(self.cx, self.cy)

    # Draw a circle at the detected object's center
    # 在检测到的物体中心绘制一个圆
    cv2.circle(result_frame, (int(self.cx), int(self.cy)), 10, (255, 255, 0), thickness=None, lineType=None, shift=None)

# Calculate and display FPS (frames per second)
# 计算并显示FPS (每秒帧数)
cur_time = time.time()
fps = str(int(1 / (cur_time - self.pr_time)))
self.pr_time = cur_time
cv2.putText(result_frame, fps, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

# Display the result image in a window
# 在窗口中显示结果图像
cv.imshow(self.windows_name, result_frame)

```

Image processing function process

```

def process(self, rgb_img, action):
    # Resize the input image to 640x480 resolution
    # 将输入图像调整为640x480分辨率
    rgb_img = cv.resize(rgb_img, (640, 480))

    # If 'r' key is pressed, reset the tracking system to initial state
    # 如果按下 'r' 键, 将跟踪系统重置为初始状态
    if action == ord('r') or action == 114:
        self.Reset()

    # State machine: Initialization state
    # 状态机: 初始化状态
    if self.Track_state == 'init':
        # Create a named window for mouse interaction (ROI selection)
        # 创建用于鼠标交互 (ROI选择) 的命名窗口
        cv.namedWindow(self.windows_name, cv.WINDOW_AUTOSIZE)
        # Set mouse callback function for selecting ROI

```

```

# 设置鼠标回调函数用于选择感兴趣区域
cv.setMouseCallback(self.windows_name, self.onMouse, 0)

if self.select_flags == True:
    # Draw a line from start point to current mouse position
    # 从起点到当前鼠标位置绘制线条
    cv.line(rgb_img, self.cols, self.rows, (255, 0, 0), 2)
    # Draw a rectangle representing the ROI
    # 绘制代表ROI的矩形
    cv.rectangle(rgb_img, self.cols, self.rows, (0, 255, 0), 2)

# State machine: Tracking state
# 状态机：跟踪状态
if self.Track_state != 'init':
    # If tracker initialization is requested
    # 如果需要初始化跟踪器
    if self.gTracker_state == True:
        Roi = (self.Roi_init[0], self.Roi_init[1],
               self.Roi_init[2] - self.Roi_init[0],
               self.Roi_init[3] - self.Roi_init[1])

        # Reinitialize the tracker with the selected ROI
        # 使用选定的ROI重新初始化跟踪器
        self.gTracker = Tracker(tracker_type=self.tracker_type)
        self.gTracker.initworking(rgb_img, Roi)
        self.gTracker_state = False # Clear the initialization flag

    # Perform tracking on the current frame
    # 对当前帧执行跟踪
    rgb_img, (targBegin_x, targBegin_y), (targEnd_x, targEnd_y) =
self.gTracker.track(rgb_img)

    # Calculate the center of the tracked object
    # 计算跟踪物体的中心
    center_x = targEnd_x / 2 + targBegin_x / 2
    center_y = targEnd_y / 2 + targBegin_y / 2

    # Update global center coordinates
    # 更新全局中心坐标
    self.cx = center_x
    self.cy = center_y
    ....
return rgb_img

```

```

# Create subscriber for voice recognition result topic
# 创建订阅语音识别结果话题的订阅者
self.sub_voice
=self.create_subscription(Int8,"voice_result",self.getVoiceResultCallback,1)

def getVoiceResultCallback(self,msg):
    if msg.data == 107:
        self.start_follow = True
        print("Start tracking .")
    elif msg.data == 76:
        self.Reset()

```

The XY_track tracking function has been explained in previous tutorials, so it will not be repeated here.